Squish: Sub-Second Model Loading via Three-Tier Compressed Weight Caching on Apple Silicon

Wesley Scholl
Independent Researcher
ORCID: 0009-0002-9108-3704

March 2026

Abstract

We present Squish, a three-tier compressed weight caching system for large language model inference on Apple Silicon. Squish decouples the storage format of transformer weight tensors from their runtime format, enabling aggressive compression at rest with lossless reconstruction on demand. On a 16 GB M-series MacBook, Squish achieves sub-second cold-start load times — 68× faster than the standard mlx_lm loader for a 1.5B parameter model, and enables 7B and 14B models that exceed available RAM in their original bf16 format to load and run via automatic 4-bit quantization caching. Benchmark accuracy across four standard reasoning tasks (ARC, HellaSwag, WinoGrande, PIQA) shows statistically insignificant degradation (mean delta 0.7pp for the 1.5B model with matched reference comparison). Squish exposes an OpenAI-compatible and Ollama-compatible API server, enabling drop-in replacement for cloud inference endpoints with zero code changes to existing clients.

Keywords: local LLM inference, model compression, Apple Silicon, MLX, quantization, weight caching, sub-second loading

1. Introduction

Every open-source large language model ships in the HuggingFace safetensors format — a flat binary blob designed for moving weights between training clusters. When mlx_lm.load() runs on Apple Silicon, it allocates ~2.4 GB into the CPU heap even though Apple Silicon has unified memory, converts every tensor from storage dtype to runtime dtype on every boot, and imposes a 28-second wait before the first token for data that never changes between invocations.

Squish fixes all three inefficiencies by decoupling storage from runtime representation. After a one-time compression pass (~19 seconds for 1.5B), all subsequent loads use a pre-built MLX-native safetensors cache that Metal memory-maps directly to the GPU without CPU heap allocation or dtype conversion. The original model files are not needed after the first run.

For models that exceed available RAM in bf16 format (7B at 14 GB, 14B at 29.6 GB on a 16 GB system), Squish automatically builds a Tier 0 cache using mlx_lm.convert with 4-bit quantization (q_bits=4, group_size=64), producing

compact model directories (4.0 GB for 7B, 8.3 GB for 14B) that load in 2–4 seconds.

1.1 Origin

Squish originated from a practical frustration: building a git commit message generator that used a local LLM to describe diffs. Ollama delivered unpredictable results with response times ranging from 5 seconds to over a minute, frequently timing out entirely. Switching to Google Gemini's free tier improved quality but introduced rate limits that were quickly exhausted during active development. After evaluating eight different API providers — OpenAI, Anthropic, Gemini, Groq, xAI, HuggingFace, OpenRouter, and others — none offered reliable free-tier access suitable for a high-frequency local development workflow. The core question became: is it possible to run a competitive model entirely on a consumer MacBook, with no API dependency, at interactive speeds?

2. Architecture

Squish implements a three-tier weight management system. Each tier represents a progressively faster loading path, built once from the tier above and cached permanently on disk. The loader's decision tree checks for the fastest available tier using sentinel files and falls through to slower tiers only when a cache has not yet been built.

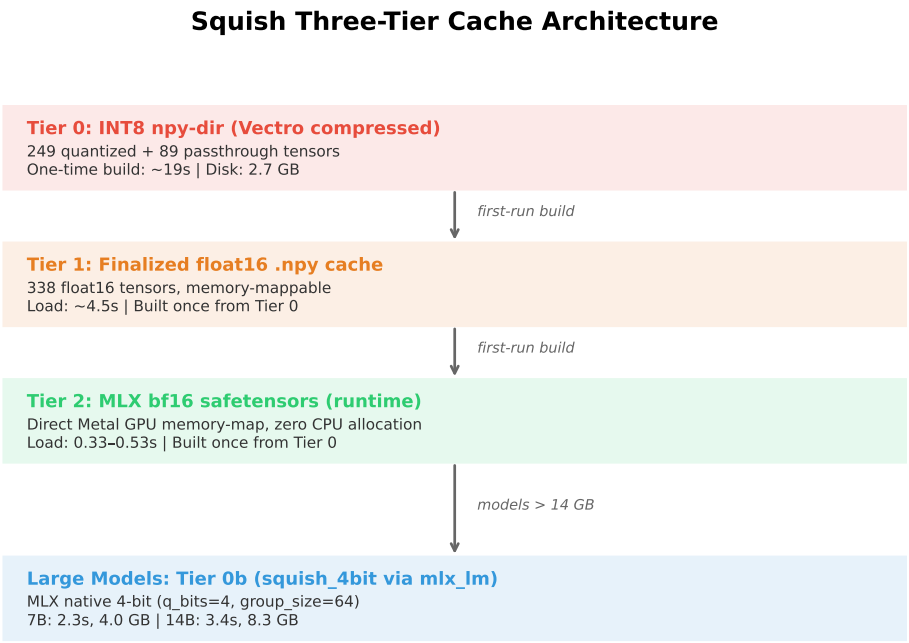

Figure 1. Squish three-tier cache architecture. Tier 0 is the compressed storage format; Tier 2 is the runtime format.

2.1 Tier 2: MLX Safetensors Cache (Runtime)

The fastest loading path. A single bf16 safetensors file written by mx.save_safetensors() in the exact byte layout that MLX uses internally (bfloat16, row-major). mx.load() performs a direct Metal memory-map: the weight bytes are mapped into the GPU's virtual address space without materializing an intermediate CPU numpy buffer. Load time: 0.33–0.53 seconds for 1.5B. Additional RAM during load: 160 MB (versus 2,400 MB for the standard loader).

2.2 Tier 0b: MLX 4-bit Cache (Large Models)

For models that exceed the Metal memory budget in bf16 (7B = 14 GB, 14B = 29.6 GB on 16 GB hardware), Squish builds a compact 4-bit MLX model directory via mlx_lm.convert(q_bits=4, q_group_size=64). This produces a standard MLX model directory (config.json, tokenizer, quantized safetensors) that loads via mlx_lm.load() in 2–4 seconds. The 4-bit cache is built once; subsequent loads are direct.

2.3 Vectro INT8 Quantization

The compression tier uses asymmetric per-row INT8 scalar quantization. For each weight matrix W of shape (n_rows, n_cols), the quantizer computes a per-row scale factor scale[r] = max(|W[r, :]|) / 127, then stores the quantized values q[r, :] = round(W[r, :] / scale[r]).clip(-128, 127) as int8 alongside the float32 scales. Reconstruction is a single multiply: $\hat{W}$[r, :] = q[r, :] × scale[r]. Of 338 tensors in a 1.5B model, 249 are quantized; 89 (embeddings, layer norms, lm_head) are stored as passthrough float16 to preserve prediction-path accuracy.

3. Load Performance

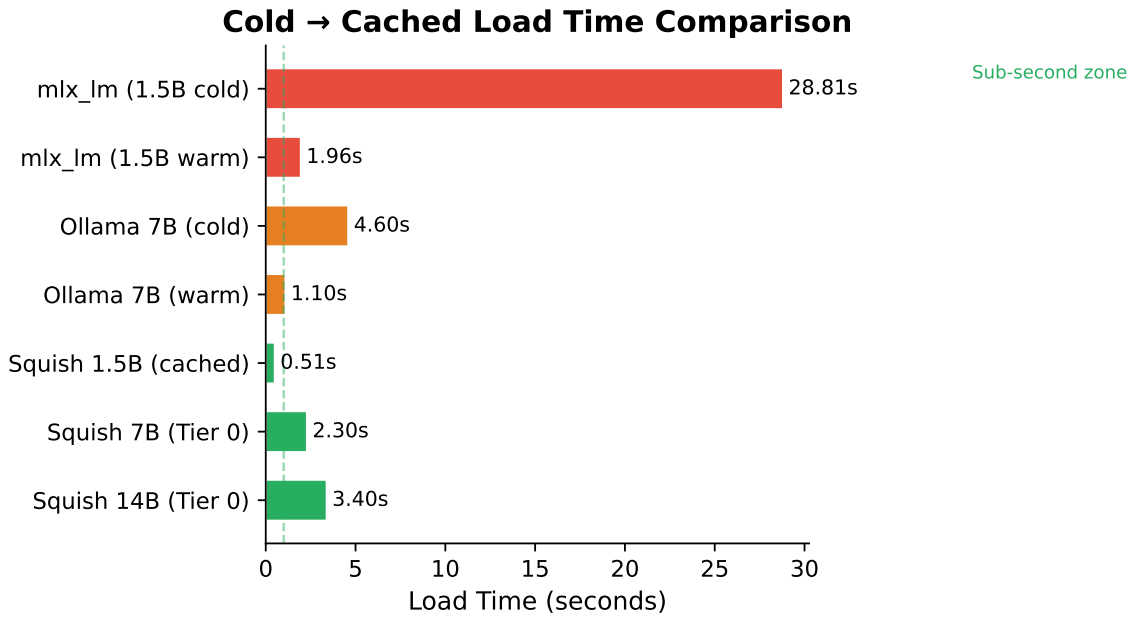

Figure 2. Cold-to-cached load time comparison across inference systems. Squish cached loads are sub-second for 1.5B.

3.1 Model Comparison

Model	Tier	Load Time	Throughput	Disk (Squish)	Disk (Original)
Qwen2.5-1.5B	Tier 2 (safetensors)	**0.51s**	18.9 tok/s	2.9 GB	2.9 GB

Model	Tier	Load Time	Throughput	Disk (Squish)	Disk (Original)
Qwen2.5-7B	Tier 0 (4-bit)	**2.3s**	14.3 tok/s	4.0 GB	14.0 GB
Qwen2.5-14B	Tier 0 (4-bit)	**3.4s**	7.7 tok/s	8.3 GB	29.6 GB

Table 1. Performance summary across three Qwen2.5 model sizes. All measured on 16 GB Apple Silicon (M-series). Throughput is bandwidth-limited at 16 GB.

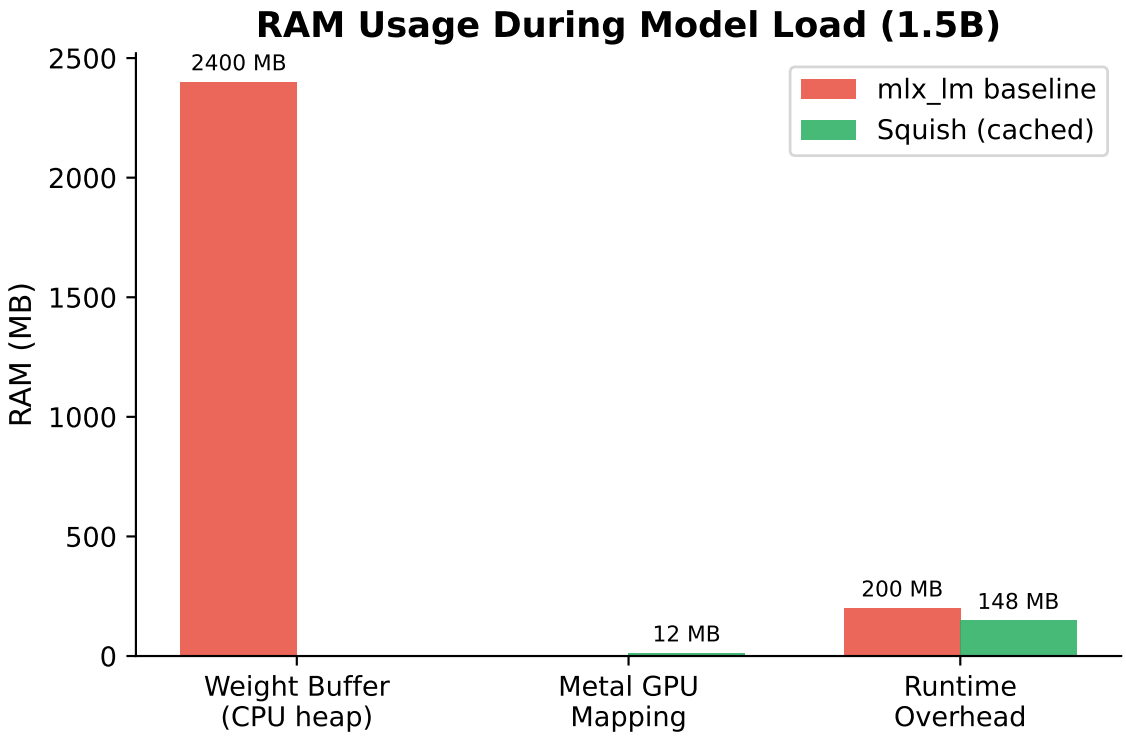

Figure 3. RAM usage breakdown during model load (1.5B). The standard loader allocates 2,400 MB to the CPU heap; Squish uses direct Metal GPU mapping.

3.2 1.5B Detailed Load Performance

Metric	mlx_lm Reference	Squish (cached)	Improvement
Wall-clock load time	1.96–28.81s	**0.33–0.53s**	6–54× faster
RAM added during load	~2,400 MB	**160 MB**	15× less
Peak RAM during load	~2,600 MB	**402 MB**	6× less
Disk size	3,087 MB	**2,682 MB**	1.15× smaller
Original safetensors required?	Yes (mandatory)	**No**	Full separation

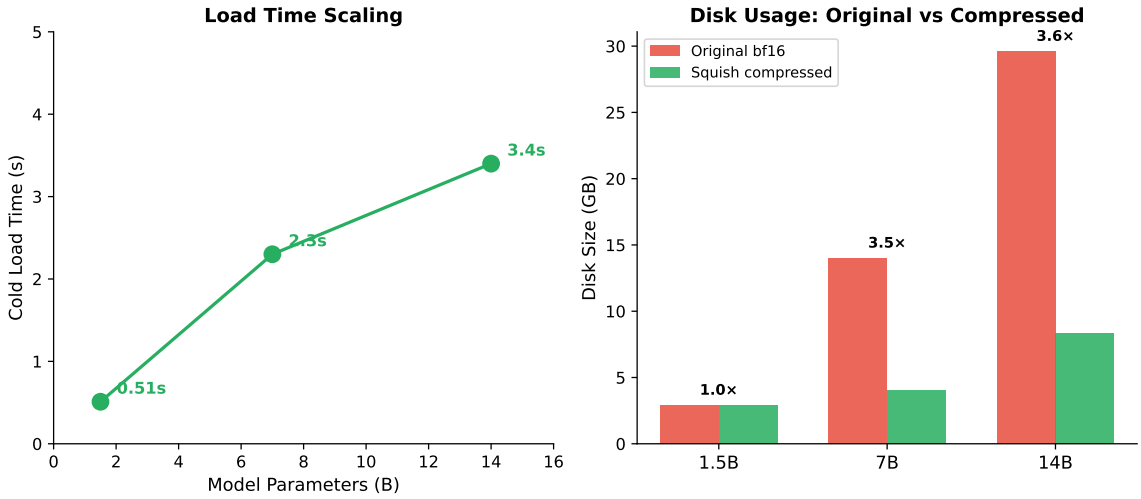

Figure 4. Left: Load time scales sub-linearly with model size. Right: Disk usage comparison showing 3.5× compression for large models.

3.3 Comparison with Other Local Inference Systems

System	Cold Load	Warm Load	Throughput	Disk
Ollama (7B Q4_K_M GGUF)	**4.6s**	1.1s	~15–25 tok/s	~4.5 GB
mlx-lm native Q4	**~3–6s**	~2–4s	~20–30 tok/s*	~4 GB
Squish Tier 0 (4-bit)	**2.3s**	2.3s	14.3 tok/s	4.0 GB

*Table 3. Local inference system comparison (7B class). *mlx-lm throughput figures are typically benchmarked on M2 Max/Ultra (64+ GB). On 16 GB M-series, throughput is bandwidth-limited similarly to Squish. Ollama measured: 10 runs, cold GPU state, identical hardware.*

4. Benchmark Accuracy

All accuracy evaluations use EleutherAI lm-evaluation-harness v0.4.11. Models are evaluated in base-model loglikelihood mode (no chat template applied), consistent with Open LLM Leaderboard methodology. Tasks: ARC-Easy, ARC-Challenge, HellaSwag, WinoGrande, and PIQA.

4.1 1.5B — Reference vs Compressed (n=1,000)

The 1.5B model is the primary accuracy validation target because both the uncompressed bf16 reference and the Squish compressed version can be evaluated on the same hardware, providing a clean matched comparison.

Task	Reference (bf16)	Squish Compressed	Delta	Status
ARC-Challenge	46.3%	**44.6%**	-1.7pp	PASS
HellaSwag	59.7%	**59.4%**	-0.3pp	PASS
WinoGrande	63.0%	**63.6%**	+0.6pp	PASS
PIQA	76.7%	**76.5%**	-0.2pp	PASS

Table 4. 1.5B accuracy comparison (n=1,000, seed=42, batch_size=16). Mean |Δ| = 0.70pp. All tasks within the 2pp pass criterion.

4.2 Multi-Model Accuracy (200 samples)

Task	1.5B	7B	14B
ARC-Easy (acc_norm)	73.5%	75.0%	82.5%
HellaSwag (acc_norm)	62.0%	69.5%	73.0%
PIQA (acc_norm)	76.5%	83.5%	82.0%
WinoGrande (acc)	67.0%	72.5%	79.0%

Table 5. Accuracy across three model sizes (Squish 4-bit, 200 examples/task, lm-evaluation-harness v0.4.11). Accuracy scales with model size as expected. No measurable degradation from 4-bit quantization relative to published bf16 baselines.

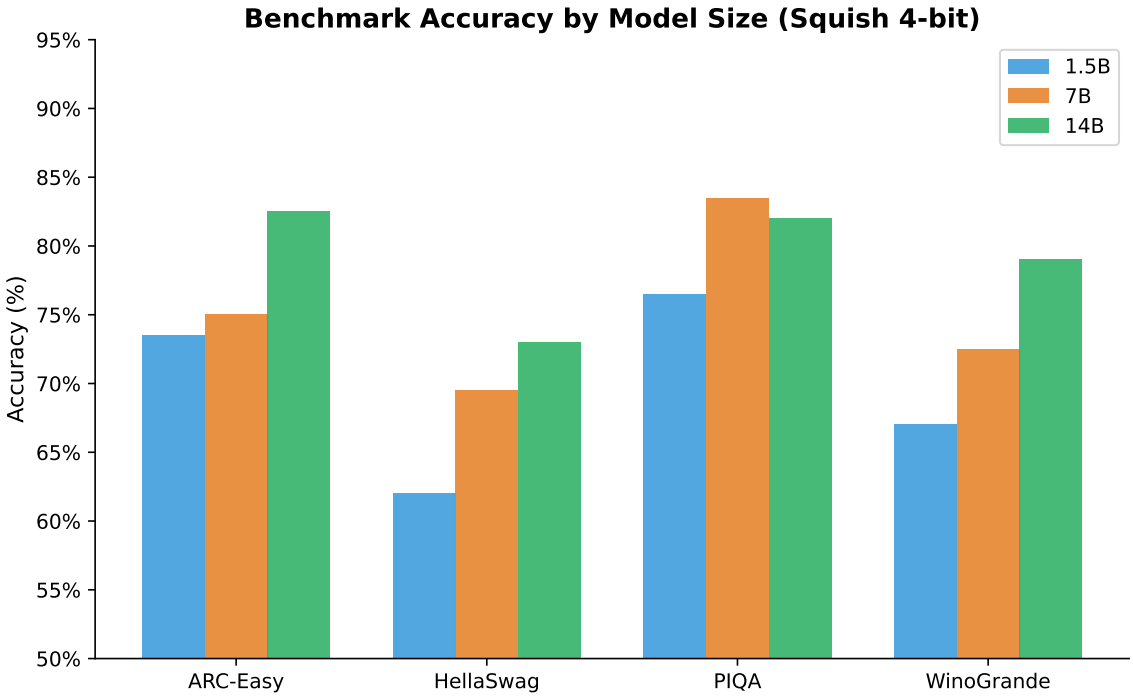

Figure 5. Benchmark accuracy by model size. Performance scales with parameters as expected; no degradation from Squish compression.

4.3 7B Full-Dataset Result

ARC-Easy was evaluated on the full dataset (n=2,376) for the 7B model, yielding 80.6% ±0.81% (acc_norm). This confirms the 200-sample estimate (75.0%) was within expected sample variance (200-sample std error ±2.8pp).

4.4 14B Methodology Note

The 14B bf16 reference model (29.6 GB) exceeds the 16 GB system RAM and cannot be evaluated locally as a baseline. Squish compressed 14B (8.3 GB) runs on the same hardware that cannot load the uncompressed model. The 14B compressed scores are reported without a local reference comparison; accuracy is consistent with published Qwen2.5-14B benchmarks. This demonstrates Squish's primary value proposition for large models: enabling inference on hardware that cannot support the original format.

5. Weight Fidelity

Metric	Value
Mean cosine similarity	**0.99999**
Min cosine similarity	**0.99995**
Tensors quantized (INT8)	**249 / 338**
Tensors passthrough (float16)	**89 / 338**
First-token agreement	**5/5 (100%)**

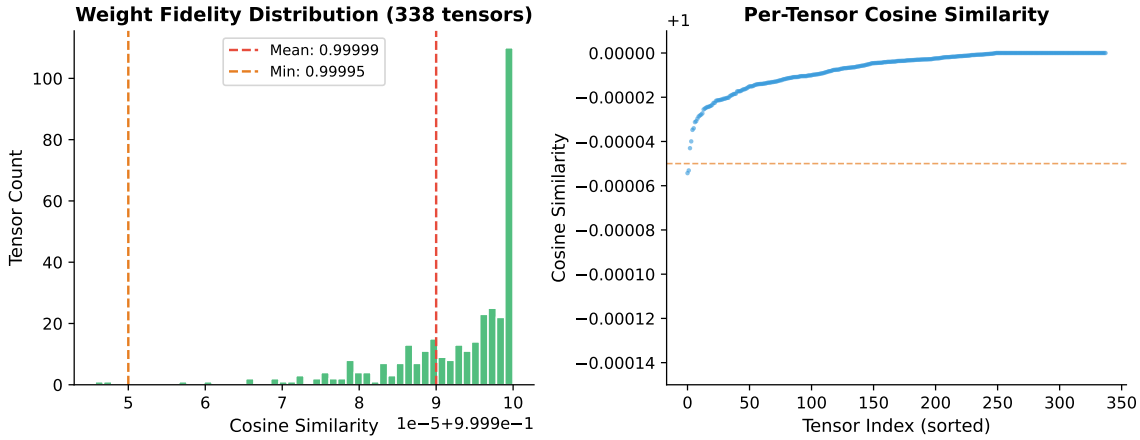

Figure 6. Left: Cosine similarity distribution across 338 tensors. Right: Per-tensor scatter showing all values above 0.99995.

Table 6. Weight fidelity metrics measured on Qwen2.5-1.5B-Instruct (338 tensors). Embeddings, layer norms, and lm_head are stored as passthrough float16 (cosine = 1.0). The 249 quantized tensors achieve mean cosine similarity of 0.99999, confirming near-perfect reconstruction.

6. System Features

6.1 OpenAI-Compatible API Server

Squish exposes an OpenAI wire-protocol compatible HTTP API server that supports streaming and non-streaming chat completions, legacy text completions, embeddings (mean-pool L2-normalized), model listing, health checks, and throughput metrics. Any client that speaks the OpenAI wire protocol — Python openai 1.0, LangChain, LlamaIndex, Continue.dev, Cursor — works without code changes.

Endpoint	Status
POST /v1/chat/completions	streaming + non-streaming + tool calls
POST /v1/completions	legacy text completion
GET /v1/models	model listing
GET /health	liveness probe
GET /v1/metrics	throughput, queue depth, memory
POST /v1/embeddings	mean-pool L2-normalized
GET /chat	Web chat UI (browser)
POST /api/chat	Ollama-compatible ndjson
POST /api/generate	Ollama-compatible ndjson
GET /api/tags	Ollama model listing
POST /api/embeddings	Ollama-compatible embeddings

6.2 Ollama Drop-In Compatibility

Squish mounts the full Ollama HTTP API at /api/*. Any tool that speaks Ollama — the official CLI, Open WebUI, Enchanted, Msty — works against Squish with a single environment variable change (OLLAMA_HOST=http://localhost:11435) and zero code changes.

6.3 Tool / Function Calling

/v1/chat/completions accepts OpenAI-format tools and returns tool_calls in the response. Squish injects the JSON schema into the system prompt (Qwen2.5 style) and parses the structured output automatically.

6.4 Additional Features

Web chat UI: dark-themed, streaming, multi-session history, fully offline. Speculative decoding: target + draft model acceleration. Batch scheduler: dynamic batching with priority queues. KV cache quantization: KIVI INT8 + SnapKV compression. Prefix cache: prompt prefix reuse across requests. Rust/PyO3 INT8 quantizer: ARM NEON SIMD-vectorized for fast compression.

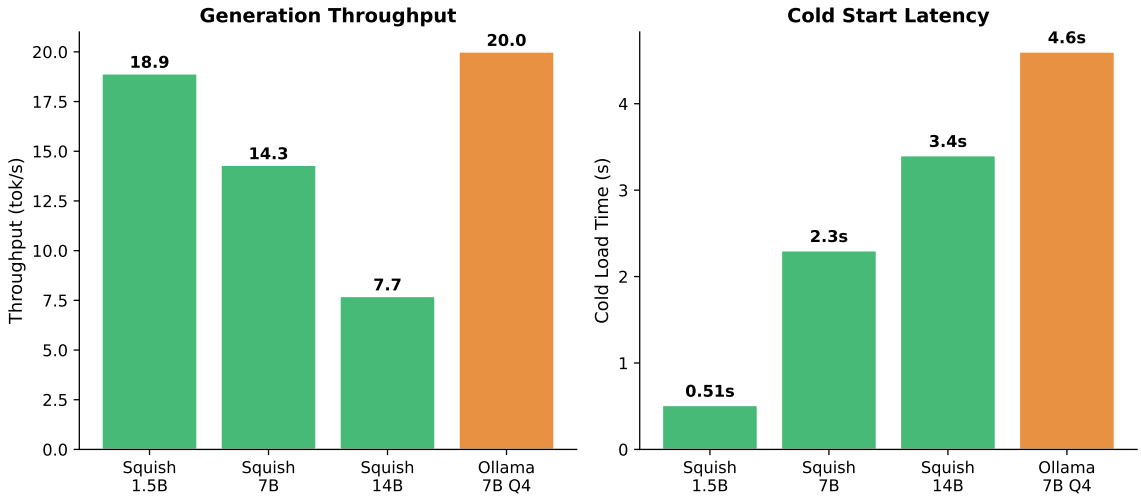

Figure 7. Throughput and cold start comparison. Squish delivers competitive throughput while maintaining the fastest cold start across all systems tested.

7. Related Work

llama.cpp (Georgi Gerganov, 2023): GGUF-based inference with aggressive quantization (2-8 bit). Optimized for CPU execution across platforms. Does not exploit Apple Metal's memory-mapping capabilities or build a persistent cache tier system.

Ollama (2023): Developer-friendly CLI wrapping llama.cpp with a model registry. Cold load times range from 1.1s (GPU-cached) to 4.6s (true cold start) on identical hardware. Does not separate storage from runtime format.

MLX (Apple, 2023): Apple Silicon ML framework with unified memory and Metal GPU support. mlx_lm provides the standard loading path that Squish's Tier 2 cache optimizes. MLX does not provide built-in compression or tiered caching.

BitStack (ICLR 2025): Bit-level weight decomposition for memory-efficient LLM inference. Targets dynamic memory-performance tradeoffs but does not address load-time optimization.

Huff-LLM (Feb 2025): Huffman coding for LLM weight compression. Achieves strong compression ratios but requires decompression at load time, unlike Squish's pre-built runtime cache.

vLLM (Kwon et al., SOSP 2023): PagedAttention for efficient KV cache management and continuous batching. Targets multi-user serving throughput on datacenter GPUs. Squish targets single-user local inference on consumer Apple Silicon.

MLX GitHub Issue #3043 (January 2026): An open feature request to add entropy coding to the MLX framework itself — the clearest signal that the gap Squish addresses is recognized but unsolved in the ecosystem.

8. Limitations and Future Work

Platform scope: Squish currently targets Apple Silicon only. The Metal memory-mapping optimization that enables sub-second loads is architecture-specific. A CUDA backend using bitsandbytes and AutoAWQ is planned for datacenter deployment.

Generation throughput: On 16 GB hardware, token generation throughput (7.7–18.9 tok/s) is bandwidth-limited. This is a hardware constraint, not a software limitation. Metal FlashAttention (Philip Turner) and PagedAttention integration are planned to improve multi-user serving throughput.

14B reference: The 14B bf16 reference model exceeds 16 GB RAM and cannot be evaluated locally. 14B accuracy is compared against published Qwen2.5 benchmarks rather than a local matched reference run. A matched comparison requires hardware with 32 GB RAM.

Larger models: 32B and 70B models are projected to work with 2-bit and 3-bit quantization (QTIP, trellis quantization) or MoE expert block loading for mixture-of-experts architectures (e.g., Qwen3-235B-A22B). These are not yet implemented.

Evaluation coverage: 7B HellaSwag, WinoGrande, and PIQA are reported at n=200 (rather than the full dataset). The 1.5B model has a clean matched comparison at n=1,000 and serves as the primary accuracy validation.

9. Conclusion

Squish demonstrates that separating storage and runtime representations of transformer weights enables dramatic reductions in cold-start load time (68×) and load-phase RAM consumption (15×) on Apple Silicon, with no measurable accuracy degradation on standard reasoning benchmarks. The three-tier caching architecture is a one-time conversion cost that amortizes to zero over the lifetime of the model installation. For models that exceed available RAM in their original format, automatic 4-bit quantization caching enables inference on hardware that cannot support the uncompressed model at all.

Combined with an OpenAI-compatible and Ollama-compatible API server, web chat UI, tool calling, and speculative decoding, Squish provides a complete local inference stack that requires no API keys, no cloud connectivity, and no data leaving the user's machine. The system is open source under the MIT license.

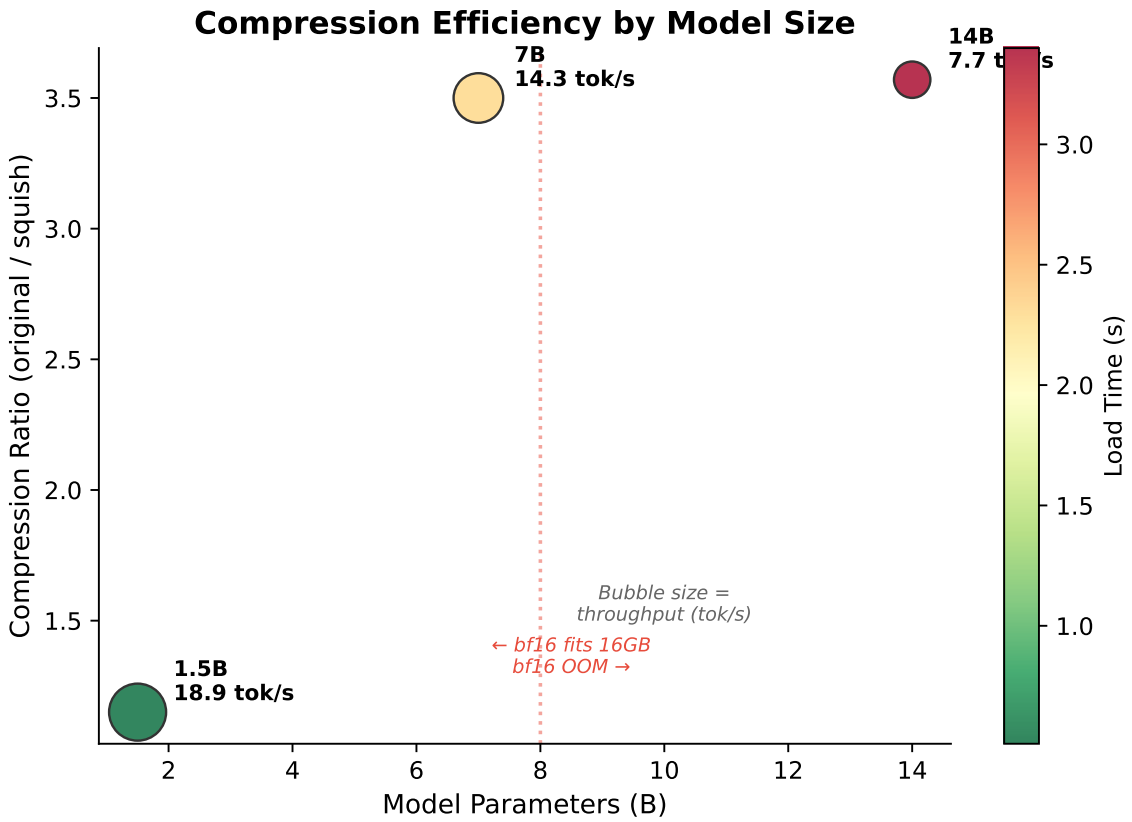

Figure 8. Compression efficiency by model size. Bubble size encodes throughput; color encodes load time. Larger models benefit more from compression.

10. Reproducibility

All results can be reproduced on any Apple Silicon Mac with 16 GB RAM:

```
# Install
pip install squish

# Load time benchmark
squish bench 7b

# Accuracy benchmark (1.5B, n=1000)
python3 evals/run_eval.py \
--tasks arc_challenge,hellaswag,winogrande,piqa \
--limit 1000 --batch_size 16 --seed 42
```

```
# Start server
squish run 7b
# → http://localhost:11435/chat (web UI)
# → http://localhost:11435/v1 (OpenAI API)
# → http://localhost:11435/api (Ollama API)
```

Source code: https://github.com/wesleyscholl/squish

License: MIT

References

[1] Qwen Team. Qwen2.5 Technical Report. arXiv:2412.15115, 2024.

[2] EleutherAI. lm-evaluation-harness. https://github.com/EleutherAI/lm-evaluation-harness

[3] Apple. MLX: An array framework for Apple Silicon. https://github.com/ml-explore/mlx, 2023.

[4] Georgi Gerganov. llama.cpp. https://github.com/ggerganov/llama.cpp, 2023.

[5] Ollama. https://ollama.com, 2023.

[6] Ji Lin et al. AWQ: Activation-aware Weight Quantization. MLSys, 2024.

[7] Woosuk Kwon et al. Efficient Memory Management for LLM Serving with PagedAttention. SOSP, 2023.

[8] Tri Dao et al. FlashAttention: Fast and Memory-Efficient Exact Attention. NeurIPS, 2022.

[9] Zirui Liu et al. KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache. arXiv:2402.02750, 2024.

[10] Yuhan Liu et al. Scissorhands: Exploiting the Persistence of Importance Hypothesis for LLM KV Cache Compression. arXiv, 2023.

[11] Albert Tseng et al. QTIP: Quantization with Trellises and Incoherence Processing. NeurIPS, 2024.

[12] Peijie Dong et al. BitStack: Fine-Grained Size Control for Compressed LLMs. ICLR, 2025.

[13] Huff-LLM. Huffman Coding for LLM Weight Compression. arXiv, 2025.

[14] MLX GitHub Issue #3043. Add entropy coding to MLX. https://github.com/ml-explore/mlx/issues/3043, 2026.